Simulating Requests

Ideally, your controllers are relatively simple. The complicated functionality is in a model or other object and is being tested in your unit tests for those objects. One reason this is a best practice is that models are easier to test than requests because they're easier to extract and use independently in a test framework.

Prescription 25	A request test should test behavior of a single request. A request test should not fail because of problems in the model.

A request test that overlaps with model behavior is part of the awkward middle ground of testing that the Rails 5 changes are designed to avoid. If the request test is going to the database, then the test is slower than it needs to be. And if a model failure can cascade into the controller tests, then it's harder than it needs to be to isolate the problem.

A request test should have one or more of the following goals:

- Verify that a normal, basic user request triggers expected model or workflow object calls. Test doubles work extremely well for this purpose.
- Verify that a side effect, such as an email or background job, is triggered as part of a user request.
- Verify that an ill-formed or otherwise invalid user request is handled properly, for whatever definition of "properly" fits your app.
- Verify security, such as requiring logins for pages as needed and testing that users who enter a URL for a resource they shouldn't be able to see are blocked or diverted. (I discuss this more in Chapter 13, *Testing for Security*, on page 261.) This is also a good candidate to be offloaded into unit testing with, for example, the Pundit gem.[2]

Your request tests in Rails 5 will usually start with a simulated request. To make this simulation easier, Rails provides a test method to simulate each HTTP verb: `delete`, `get`, `head`, `patch`, `post`, and `put`. Each of these methods works the same way. (Internally, they all dispatch to a common method that does all the work.) A full call to one of these methods has one argument and up to five optional keyword arguments. The argument is a URL string, which can be passed as a string or as a Rails URL helper. The keyword arguments are, alphabetically:

2. https://github.com/elabs/pundit